

■ Lab 2 — Resource Block : EC2 & Security Group (AWS CloudShell)

■ Objectifs du Lab

À la fin de ce lab, vous aurez :

- Compris la structure d'un **resource block** Terraform
- Déployé une **instance EC2** et un **Security Group sans règles** sur AWS
- Utilisé le workflow complet : `terraform init` → `validate` → `fmt` → `plan` → `apply` → `destroy`

■ Étape 1 — Préparer l'environnement

Ouvrez **AWS CloudShell** depuis la console AWS (icône >_ en haut à droite).

Si Terraform n'est pas disponible (session redémarrée), réinstallez-le :

```
bash $HOME/install-terraform.sh
```

Placez-vous dans le dossier du lab :

```
cd $HOME/terraform-training/lab02-basics
```

Créez les fichiers du lab :

```
touch versions.tf main.tf outputs.tf
```

■ Étape 2 — Bloc `terraform` et `provider`

Ouvrez l'éditeur CloudShell :

```
nano versions.tf
```

Collez le contenu suivant :

```
terraform {  
  required_version = ">= 1.15.0"  
  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = "eu-west-1"  
}
```

Sauvegardez : **Ctrl+O** → **Entrée** → **Ctrl+X**

■ Dans CloudShell, les credentials AWS sont **automatiquement configurés** — pas besoin d'ajouter d'authentification dans le provider block.

■ Étape 3 — Security Group sans règles

```
nano main.tf
```

```
resource "aws_security_group" "lab2_sg" {  
  name        = "lab2-security-group"  
  description = "Security group sans regles - Lab 2"  
  
  tags = {  
    Name = "lab2-sg"  
    Lab  = "lab2"  
  }  
}  
  
resource "aws_instance" "lab2_instance" {  
  ami                = "ami-0694d931cee176e7d"  
  instance_type      = "t2.micro"  
  vpc_security_group_ids = [aws_security_group.lab2_sg.id]  
  
  tags = {  
    Name = "lab2-instance"  
    Lab  = "lab2"  
  }  
}
```

Sauvegardez : **Ctrl+O** → **Entrée** → **Ctrl+X**

■ Étape 4 — Outputs

```
nano outputs.tf
```

```
output "instance_id" {
  description = "ID de l'instance EC2"
  value      = aws_instance.lab2_instance.id
}

output "instance_public_ip" {
  description = "IP publique de l'instance"
  value      = aws_instance.lab2_instance.public_ip
}

output "security_group_id" {
  description = "ID du Security Group"
  value      = aws_security_group.lab2_sg.id
}

output "security_group_name" {
  description = "Nom du Security Group"
  value      = aws_security_group.lab2_sg.name
}
```

Sauvegardez : **Ctrl+O** → **Entrée** → **Ctrl+X**

■ Étape 5 — Workflow Terraform

```
# Initialiser
terraform init

# Valider
terraform validate

# Formater
terraform fmt

# Planifier
terraform plan

# Appliquer
terraform apply
```

Tapez **yes** pour confirmer l'apply.

```
# Voir les outputs
terraform output
```

■ Étape 6 — Vérification sur AWS

```
aws ec2 describe-instances \
  --filters "Name=tag:Lab,Values=lab2" \
  --query "Reservations[*].Instances[*].[InstanceId,State.Name,PublicIpAddress]" \
  --output table

aws ec2 describe-security-groups \
  --filters "Name=tag:Lab,Values=lab2" \
  --query "SecurityGroups[*].[GroupId,GroupName]" \
  --output table
```

■ Étape 7 — Nettoyage

```
terraform destroy
```

Tapez **yes** pour confirmer.

■■ Si votre session CloudShell a redémarré entre temps, relancez ``bash $HOME/install-terraform.sh`` avant de lancer ``terraform destroy``.

■ Validation du Lab

#	Critère	Validé
1	<code>`terraform init`</code> réussi, provider AWS téléchargé	■
2	<code>`terraform validate`</code> retourne "Success"	■
3	<code>`terraform plan`</code> affiche 2 ressources à créer	■
4	<code>`terraform apply`</code> crée l'instance EC2 et le SG	■

#	Critère	Validé
5	`terraform output` affiche l'instance ID et le SG	■
6	`terraform destroy` supprime les 2 ressources	■

■ ****Fin du Lab 2 — Première infrastructure déployée avec Terraform !****

*Le Lab 3 explore la gestion du ****State file**** : `terraform state list`, `show`, `mv` et `refresh`.*